

UnityChan RPG

Level: 11

장비 착용

메구미

우선 이상단에 있

1 2 3 4 5 6 다음

프로젝트	개인 프로젝트
담당 역할	UI 관리, 전투와 몬스터 행동, 퀘스트·상점·강화, 데이터 관리
기술	Unity, C#, JSON, CSV, ScriptableObject, Shader Graph
기간	2025.08.04 ~ 2025.11.11

01 / 50

내가 구현한 영역

UI 관리, 전투와 몬스터 행동, 퀘스트·상점·강화, 데이터 관리

Unity로 만든 개인 RPG 프로젝트다. 플레이어 전투, 미노타우르스와 늑대의 이동, 퀘스트·대화, 상점·강화와 UI를 구현했다. 미노타우르스는 방향별 관심도를 사용하는 CBS로, 늑대는 후보 지점 평가와 예약을 사용하는 EQS 형태로 이동을 구성한다.

대사는 CSV의 노드와 연결 표로 관리한다. 아이템 정의와 플레이 진행 정보는 JSON으로 읽고 저장하며 스킬·퀘스트·강화 이펙트는 ScriptableObject 에셋을 실행 코드에 연결한다.

담당 구현

- **UI:** 팝업 스택과 z-order, 클릭 포커스, 상대 좌표 드래그, 아이콘 교환과 퀵 슬롯 등록.
- **게임 시스템:** 퀘스트 목표 평가, 다이얼로그 연결, 상점 거래와 장비 강화.
- **몬스터:** 미노타우르스 CBS, 늑대 EQS, NavMesh 기반 스폰 셀과 박스 Gizmo.
- **데이터와 표현:** CSV 파싱, JSON 직렬화, SO 정의와 이벤트 실행, Shader Graph 디졸브.

기술 구성

Unity, C#, JSON, CSV, ScriptableObject, Shader Graph

실행 화면과 구현 코드에 연결된 블로그·영상 링크는 PDF에서 클릭해 열 수 있습니다.

기능별 구현

01	드래그 앤 드롭과 킥 슬롯 교환	04
02	UI 관리와 팝업 z-order	07
03	미노타우르스의 Context Based Steering	11
04	능대의 EQS 후보 평가와 위치 예약	14
05	CSV 관리와 대화 노드 연결	18
06	퀘스트·다이얼로그와 지역 방문 이벤트	21
07	상점 목록과 아이템 구매·판매	26
08	무기 강화 수치와 속성 이펙트	29
09	ScriptableObject로 스킬·퀘스트·이펙트 관리	34
10	JSON 정의 데이터와 플레이 진행 저장	42
11	스포너 셀 배치와 박스 Gizmo	45
12	Shader Graph 디졸브와 C# 제어	49

01 / UI와 상호작용

드래그 앤 드롭과 퀵 슬롯 교환

드래그 좌표 변환, 아이콘 복사, 슬롯 교환과 SkillSO 핸들 등록 과정을 구현했다.

[블로그 원문 ↗](#)

Drag and Drop Swap · 퀵 슬롯 사이의 아이콘 교환 · [YouTube에서 보기](#)

Skill2QuickSlot · 스킬북에서 퀵 슬롯으로 등록 · [YouTube에서 보기](#)

아이콘 이동과 슬롯 배치

`UI_Draggable_Move`는 포인터 이동을 처리하고, `UI_Droppable`은 놓인 아이콘을 슬롯에 배치한다. 인벤토리·스킬북의 원본 위치는 `OriginTransform`, 직전에 배치된 슬롯은 `PreviousTransform`으로 보관한다. `IsPrevOriginal`은 원본에서 가져온 아이콘인지 슬롯 사이에서 옮기는 아이콘인지 구분한다.

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/  
├─ UI/Common/DrafAndDrop/UI_Draggable_Move.cs  
├─ UI/Common/DrafAndDrop/UI_Droppable.cs  
└─ UI/Skills/QuickSlot/UI_QuickSlot.cs
```

TEXT

```
포인터 누름 → 드래그 Canvas로 이동 → 슬롯 OnDrop  
→ 아이콘 데이터 복사·교환 → 슬롯 키 갱신
```

드래그 Canvas에서 좌표 계산

드래그를 시작하면 오브젝트의 부모를 `Draggable_Canvas`로 바꾸고 마지막 자식으로 배치한다. `CanvasGroup.alpha`를 0.6으로 낮추고 `blocksRaycasts`를 끈다. 이동 중인 아이콘 아래의 슬롯이 드롭 이벤트를 받을 수 있도록 하는 설정이다.

화면 좌표는 `ScreenPointToLocalPointInRectangle`로 드래그 Canvas의 로컬 좌표로 변환한다. Overlay Canvas에는 카메라로 `null`을 전달한다. 아이콘의 오프셋은 0이고, 팝업을 움직일 때는 눌렀던 지점과 팝업 위치의 차이를 더한다.

드래그 앤 드롭과 퀵 슬롯 교환 · 계속

Assets/04.Scripts/UI/Common/DrafAndDrop/UI_Draggable_Move.cs 발췌

C#

```
public void OnDrag(PointerEventData eventData)
{
    if (_dragPlane != null && RectTransformUtility.ScreenPointToLocalPointInRectangle(
        _dragPlane, eventData.position, _dragCamera, out Vector2 pointerPoint))
    {
        rect.localPosition = (Vector3)pointerPoint + _pointerOffset;
    }
}
```

원본 복사와 슬롯 교환

OnDrop은 ItemIcon_Prefab을 대상으로 동작한다. 풀에서 아이콘을 빌린 다음 일반 아이템은 UpdateItemData, 스킬은 UpdateSkillSOData로 데이터를 넘긴다. 원본 아이콘은 원래 부모로 돌려놓고, 새 아이콘에 드래그 상태를 복사한다.

Assets/04.Scripts/UI/Common/DrafAndDrop/UI_Droppable.cs 발췌

C#

```
ItemDataStorage copyTargetIDS = eventData.pointerDrag.GetComponentInChildren<ItemDataStorage>();
eITEMTYPE type = copyTargetIDS.GetItemtype();
switch(type)
{
    case eITEMTYPE.SKILL:
        itemGO.GetComponentInChildren<ItemDataStorage>().UpdateSkillSOData(copyTargetIDS.GetSkillSO());
        break;
    default:
        itemGO.GetComponentInChildren<ItemDataStorage>().UpdateItemData(copyTargetIDS);
        break;
}
```

목적지 슬롯에 아이콘이 있으면 IsPrevOriginal로 처리 경로를 나눈다. 원본에서 새로 등록하는 경우 기존 슬롯 아이콘을 풀에 돌려준다. 다른 슬롯에서 옮기는 경우 목적지의 기존 아이콘을 PreviousTransform으로 보내 두 슬롯의 내용을 교환한다.

배치가 끝나면 FixDropItem으로 아이콘 크기와 위치를 맞춘다. 이어서 EmptySlotKey와 SetSlotKey를 호출해 화면 배치와 입력 키 등록을 함께 갱신한다. 슬롯 밖에 놓으면 OnEndDrag가 복귀와 DropEvent 정리를 처리한다.

스킬북에서 퀵 슬롯으로 등록

퀵 슬롯은 SkillSO.handle과 슬롯 번호를 연결한다. 스킬 아이콘에 저장한 SO 참조에서 핸들을 읽고 UI_DisplayQuickSlots.AddKey에 전달한다. 일반 아이템은 ItemInfo를 키로 전달한다.

드래그 앤 드롭과 퀵 슬롯 교환 · 계속

Assets/04.Scripts/UI/Skills/QuickSlot/UI_QuickSlot.cs 발췌

CSHARP

```
public override void SetSlotKey(ItemDataStorage ids)
{
    switch(ids.GetItemTypeInfo())
    {
        case eITEMTYPE.SKILL:
            GetComponentInParent<UI_DisplayQuickSlots>().AddKey(ids.GetSkillSO().handle, __slotNO);
            break;
        default:
            GetComponentInChildren<UI_DisplayQuickSlots>().AddKey(ids.ItemInfo, __slotNO);
            break;
    }
}
```



퀵 슬롯에 등록된 아이콘의 위치를 교환하는 장면



스킬북과 퀵 슬롯을 함께 표시한 등록 화면

02 / UI와 상호작용

UI 관리와 팝업 z-order

팝업 스택과 Canvas.sortingOrder를 연결하고 클릭 포커스와 드래그 상대 좌표를 관리했다.

[블로그 원문 ↗](#)

UI 객체와 열린 순서 관리

UIManager는 @UI_Root 아래에 UI를 배치한다. _cachedUIObjects에는 이름으로 다시 찾을 UI를, _openUIObjects에는 열린 UI를 등록한다. 팝업의 앞뒤 순서는 Stack<UI_Popup>에 보관한다.

ShowPopupUI는 풀에서 팝업을 가져와 열린 목록과 스택에 추가한다. 팝업의 Canvas에 _zOrder를 할당한 뒤 값을 증가시킨다. 순서 기준은 enum으로 구분한다.

Assets/04.Scripts/Managers/UISManager/UISManager.cs 발췌

```
CSHARP

public enum UI_SortOrder
{
    WorldUIBase = 0,
    SceneUIBase = 1,
    PopUpUIBase = 10,
    DraggableUIBase = 1000
}
```

디렉터리와 실행 흐름

```
TEXT

Assets/04.Scripts/
├ Managers/UISManager/UISManager.cs
├ Managers/UISManager/UIPopUp/UI_Popup.cs
└ UI/Common/DrafAndDrop/UI_Draggable_Move.cs
```

```
TEXT

팝업 열기 → 열린 UI 등록 → 스택 Push → sortingOrder 할당
팝업 클릭 → Raycast 판정 → 스택 재배치 → 최상단 Canvas 순서 반영
```

클릭한 팝업을 가장 앞으로 이동

UpdateInputFocus는 한 프레임의 왼쪽 클릭을 한 번 판정한다. EventSystem.RaycastAll 결과 중 첫 GraphicRaycaster 결과에서 부모 UI_Popup을 찾는다. 팝업을 클릭하면 BringPopupToFront를 호출한다. 드래그 컴포넌트의 OnPointerDown도 같은 메서드를 사용한다.

UI 관리와 팝업 z-order · 계속

[Assets/04.Scripts/UI/Common/DrafAndDrop/UI_Draggable_Move.cs](#) 발췌

C#

```
public void OnPointerDown(PointerEventData eventData)
{
    if (eventData.button == PointerEventData.InputButton.Left)
        Managers.UI.BringPopupToFront(GetComponentInParent<UI_Popup>());
}
```

`BringPopupToFront`는 선택한 팝업을 제외하고 기존 팝업을 아래부터 다시 쌓는다. `sortingOrder`를 `PopUpUIBase`부터 순서대로 할당한 다음 선택한 팝업에 가장 큰 값을 지정한다. 마지막으로 스택의 맨 위와 형제 순서의 마지막에 배치한다.

[Assets/04.Scripts/Managers/UISystem/UISystem.cs](#) 발췌

C#

```
public void BringPopupToFront(UI_Popup popup)
{
    if (popup == null || !popup.gameObject.activeInHierarchy || !_popupStack.Contains(popup)) return;
    _focusedPopup = popup;
    if (_popupStack.Peek() == popup) return;
    UI_Popup[] popups = _popupStack.ToArray();
    _popupStack.Clear();
    _zOrder = (int)UI_SortOrder.PopUpUIBase;
    for (int i = popups.Length - 1; i >= 0; i--)
    {
        UI_Popup current = popups[i];
        if (current == null || current == popup) continue;
        current.GetComponent<UnityEngine.Canvas>().sortingOrder = _zOrder++;
        _popupStack.Push(current);
    }
    popup.GetComponent<UnityEngine.Canvas>().sortingOrder = _zOrder++;
    _popupStack.Push(popup);
    popup.transform.SetAsLastSibling();
}
```

마우스와 팝업 사이의 상대 위치 유지

팝업은 드래그 시작 지점과 `RectTransform` 위치의 차이를 `_pointerOffset`에 저장한다. 드래그 중에는 변환한 포인터 위치에 이 차이를 더한다. 따라서 제목 표시줄의 어느 지점을 잡았는지에 따라 팝업과 마우스의 상대 위치가 유지된다.

UI 관리와 팝업 z-order · 계속

[Assets/04.Scripts/UI/Common/DrafAndDrop/UI_Draggable_Move.cs](#) 발췌

CSHARP

```
public void OnBeginDrag(PointerEventData eventData)
{
    Canvas dragCanvas = Managers.UI.GetCachedUIByName("Draggable_Canvas").GetComponentInParent<Canvas>();
    canvas = dragCanvas.transform;
    _dragPlane = canvas as RectTransform;
    _dragCamera = dragCanvas.renderMode == RenderMode.ScreenSpaceOverlay ? null : dragCanvas.worldCamera;
    Managers.UI.SetDragging(this, true);
    transform.SetParent(canvas);
    transform.SetAsLastSibling();
    canvasGroup.alpha = 0.6F;
    canvasGroup.blocksRaycasts = false;
    _pointerOffset = Vector3.zero;
    if (_isPopupUI && RectTransformUtility.ScreenPointToLocalPointInRectangle(
        _dragPlane, eventData.pressPosition, _dragCamera, out Vector2 pressPoint))
    {
        _pointerOffset = rect.localPosition - (Vector3)pressPoint;
    }
    OnDrag(eventData);
}
```

드래그가 끝나면 팝업을 원래 부모로 옮기고 투명도와 Raycast 설정을 복원한다. [BringPopupToFront](#)를 다시 호출해 팝업 스택의 순서도 맞춘다.

UI 포커스와 게임 입력

[_activeDrags](#)는 진행 중인 드래그를 집합으로 보관한다. [BlocksGameplayInput](#), [BlocksAttackInput](#), [BlocksCameraInput](#)은 모달 팝업, 포커스, 드래그, 텍스트 입력 여부를 조합한다. UI에서 시작한 왼쪽 클릭은 버튼을 놓을 때까지 소비하도록 기록한다.

[Assets/04.Scripts/Managers/UISystem/UIManager.cs](#) 발췌

CSHARP

```
public bool BlocksAttackInput
{
    get
    {
        UpdateInputFocus();
        return HasModalPopup || IsFocusedPopupActive || IsDragging ||
            HasTextInputFocus() || _consumeLeftClickUntilRelease;
    }
}
```

UI 관리와 팝업 z-order · 계속



상점·아이템 상세·거래 확인 팝업이 함께 열린 화면

03 / 전투와 행동

미노타우르스의 Context Based Steering

목표 거리로 이동 상태를 정하고 방향별 관심도와 장애물 감쇠를 합산해 이동 벡터를 계산했다.

[블로그 원문 ↗](#)

Cow CBS · 미노타우르스의 접근과 전투 이동 · [YouTube에서 보기](#)

전투 중 방향 계산

미노타우르스의 전투 이동은 `MonsterBattleScript.UpdateBattle`에서 갱신한다. 플레이어까지의 벡터를 XZ 평면으로 투영하고 거리와 방향을 계산한다. `CalculateInterests`가 이동 방향을 반환하면 `UpdateMovement`가 이동과 회전을 적용하고, `ChooseAnimation`이 이동 상태에 맞는 Animator 값을 정한다.

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/  
├ Monsters/Minotaur/MonsterBattleScript.cs  
└ Monsters/MonsterWolf/TreeNodees/MonsterBattleAction.cs
```

TEXT

목표 거리 → 전진·후퇴·횡이동 선택 → 방향별 관심도
→ Raycast 감쇠 → 이동 벡터 → 위치·회전·애니메이션

거리로 전진·후퇴·횡이동 선택

`_rangeOffset`보다 가까우면 `MoveB`, 그 값에서 2만큼 더 먼 범위까지는 `Strafe`, 바깥은 `MoveF`를 선택한다. 한 번 선택한 상태를 모든 방향 샘플에 적용한다.

기본 방향 수는 16이다. 각 방향은 `transform.forward`를 Y축으로 회전해 만든다. 목표 방향과 샘플 방향의 내적을 `d`라고 하면 전진 관심도는 $0.5 \times (1 + d)$, 후퇴 관심도는 $0.5 \times (1 - d)$, 횡이동 관심도는 $2 \times (1 - |d|)$ 로 계산한다.

미노타우르스의 Context Based Steering · 계속

Assets/04.Scripts/Monsters/Minotaur/MonsterBattleScript.cs 발췌

CSHARP

```

_nowState = lengthToTargetOnXZ < _rangeOffset ? eNowState.MoveB
    : lengthToTargetOnXZ <= _rangeOffset + 2f ? eNowState.Strafe
    : eNowState.MoveF;
if (isFirst) _nowLookDir = transform.forward;
Vector3 interestSum = Vector3.zero;
for (int i = 0; i < _numRays; i++)
{
    Vector3 axis = Quaternion.Euler(0, i * 360f / _numRays, 0) * transform.forward;
    float dotProduct = Vector3.Dot(toTarget, axis);
    float interest = _nowState == eNowState.MoveB ? 0.5f * (1f - dotProduct)
        : _nowState == eNowState.Strafe ? 2f * (1f - Mathf.Abs(dotProduct))
        : 0.5f * (1f + dotProduct);
}

```

장애물 감지와 벡터 합산

각 방향으로 Raycast를 쏘고 `Enemies`, `Obstacles` 레이어를 검사한다. 충돌 지점까지의 거리에서 캡슐 반지름을 뺀 비율을 안전도에 사용한다. 관심도에 안전도를 곱한 뒤 방향 벡터에 가중치를 주어 합산한다.

Assets/04.Scripts/Monsters/Minotaur/MonsterBattleScript.cs 발췌

CSHARP

```

Ray ray = new Ray(transform.position + _offset, axis);
_dangerMap[i] = 0f;
if (Physics.Raycast(ray, out RaycastHit hit, _lengthRay, _obstacleMask,
    QueryTriggerInteraction.Ignore) && !hit.transform.IsChildOf(transform))
{
    float safety = Mathf.Clamp01((hit.distance - _capsuleR)
        / Mathf.Max(0.001f, _lengthRay - _capsuleR));
    interest *= safety;
    _dangerMap[i] = 1f - safety;
}
_interestMap[i] = Mathf.Max(0f, interest);
_debugLines[i].ray = new Ray(ray.origin, _interestMap[i] * axis);
_debugLines[i].isInterested = _dangerMap[i] <= 0f;
interestSum += _interestMap[i] * axis;
}

```

전진과 후퇴는 합산 벡터를 정규화해 사용한다. `Strafe` 상태에서는 `Strafe(toTarget)`가 목표 방향을 ± 90 도 회전한 좌우 방향 중 하나를 반환한다. 이 경로는 전진·후퇴의 합산 결과와 구분된다. 속도는 횡이동 때 기본 속도의 0.3배, 후퇴 때 0.5배로 조정한다.

이동 방향과 바라보는 방향

`UpdateBattle`은 플레이어를 향하는 `_nowLookDir`를 `Vector3.RotateTowards`로 갱신한다. `UpdateMovement`는 이동 벡터를 위치에 더하고 바라보는 방향에는 `Quaternion.Slerp`를 적용한다. 횡이동하면서도 회전 목표는 플레이어 방향으로 유지한다.

Assets/04.Scripts/Monsters/Minotaur/MonsterBattleScript.cs 발췌

CSHARP

```
public virtual void UpdateMovement(Vector3 direction, Vector3 lookDir)
{
    if (direction.sqrMagnitude < 0.01f) return;
    {
        transform.position += direction * _monsterSpeed * Time.deltaTime;
        if (direction != Vector3.zero)
        {
            Quaternion targetRotation = Quaternion.LookRotation(lookDir);
            transform.rotation = Quaternion.Slerp(transform.rotation, targetRotation, Time.deltaTime * 5f);
        }
    }
}
```



미노타우르스의 플레이어 주변 전투 장면

04 / 전투와 행동

늑대의 EQS 후보 평가와 위치 예약

플레이어 주변 후보 지점을 평가하고 예약·점유 상태를 갱신하며 늑대의 이동 목적지를 정했다.

[블로그 원문 ↗](#)

Wolf EQS 모방 · 여러 늑대의 위치 선택과 전투 · [YouTube에서 보기](#)

플레이어 주변 후보 지점

늑대는 EQS 형태의 후보 평가 시스템으로 이동할 위치를 선택한다. `EQSManager`가 링별 후보 지점과 전체 목록을 관리하고, `EQSQuery`가 테스트 점수를 계산한다. `EQSQuerierBehaviour_Battle`은 선택한 지점의 예약과 실제 이동을 처리한다.

`EQSPoint`에는 위치, 링 번호, 링 안의 인덱스, 예약자와 점유자를 저장한다. 이동 중 예약과 도착 후 점유를 별도 필드로 표현한다.

`Assets/04.Scripts/Managers/SlotManager/EQSManager.cs` 발췌

C#

```
public class EQSPoint
{
    public Vector3 pos { get; set; }
    public Vector3 toTargetDir { get; set; }
    public EQSQuerier occupant { get; set; }
    public EQSQuerier reserver { get; set; }
    public int ringIndex { get; set; }
    public int pointIndexInRing { get; set; }
    public bool isAvailable => (null == occupant && null == reserver);
    public bool isOccupied => (null != occupant);
    public bool isDifferntPointInSameRing(EQSPoint other) =>
        (other.ringIndex == this.ringIndex) && (other.pointIndexInRing != this.pointIndexInRing);
    public float DeltaTheta(EQSTarget target)
    {
        var vecDir = this.pos - target.transform.position;
        vecDir.y = 0;
        return Mathf.Atan2(vecDir.z, vecDir.x);
    }
}
```

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/
├─ Managers/SlotManager/EQSManager.cs
├─ CommonScripts/EQSSystem/EQSQuery.cs
└─ CommonScripts/EQSSystem/EQSBehavior/EQSQuerierBehaviour_Battle.cs
```

늑대의 EQS 후보 평가와 위치 예약 · 계속

TEXT

플레이어 주변 후보 → 테스트 점수 합산 → 후보 선택
→ 구역 검사 → 위치 예약 → NavMesh 이동 → 위치 점유

테스트 조합으로 점수 계산

각 테스트는 `IEQSTest.Execute(point, querier, context)`로 후보를 평가한다. 이상적 거리, 시야, 점유, 간격, 방향 성향 등을 SO로 정의하고 `EQSQuery`의 테스트 목록에 연결한다. 테스트 원본은 `ScriptableObject.Instantiate`로 복사해 쿼리에 보관한다.

가중합 정책은 각 점수에 `Weight / totalWeight`를 곱해 합산한다. 쿼리는 가중 곱과 최솟값 정책도 제공한다. 평가 결과는 내림차순으로 정렬하고 `_topResultCount`개 중 무작위로 하나를 고른다. 기본 후보 수가 1이면 가장 높은 점수의 지점을 고른다.

`Assets/04.Scripts/CommonScripts/EQSSystem/EQSQuery.cs` 발췌

C#

```
case SCORING_POLICY.eWeightedSum:
{
    float sum = 0f;
    foreach(var testWrapped in _dicTestConfig)
    {
        var test = testWrapped.Value.GetTest();
        float score = test.Execute(point, querier, context);
        sum+= score* (test.Weight / totalWeight);
    }
    return sum;
}
```

점유 테스트는 자신의 점유 지점에 1을 반환하고 다른 늑대가 점유하거나 예약한 지점에는 음의 무한대를 반환한다. 비어 있는 지점에는 늑대와 거리와 점수 곡선을 적용한다.

`Assets/04.Scripts/CommonScripts/EQSSystem/EQSTests/OccupancyTest/EQSOccupancyTestSO.cs` 발췌

C#

```
public float Execute(EQSPoint point, EQSQuerier querier, EQSContext context)
{
    if (point.occupant == querier)
        return 1.0f;
    if (point.occupant != null && point.occupant != querier)
        return float.NegativeInfinity;
    if (point.reserver != null && point.reserver != querier)
        return false ? 0f : float.NegativeInfinity;
    Vector3 vector = point.pos - querier.transform.position;
    vector.y=0;
    float distXZ = Vector3.Magnitude(vector);
    float score = distXZ/_maxDistance;
    return _scoreCurve.Evaluate(Mathf.Clamp01(1-score));
}
```

늑대의 EQS 후보 평가와 위치 예약 · 계속

후보 점수의 Gizmo 표시

`EQSQuery.OnDrawGizmos()`는 평가 결과를 보관한 `scoredPoints`를 순회하며 후보 위치에 반지름 0.1의 와이어 구를 그린다. 점수가 0 이하이면 파란색으로 표시하고, 양수이면 $1 - \text{score}$ 를 빨간 채널에, `score`를 초록 채널에 적용한다. 후보 지점의 배치와 평가 점수를 Scene 뷰에서 함께 확인하는 구조다.



플레이어 주변의 EQS 후보 지점과 늑대 배치를 표시한 디버그 화면

`Assets/04.Scripts/CommonScripts/EQSSystem/EQSQuery.cs` 발췌

```
CSHARP

float score = point.score;
if (0f >= score)
    Gizmos.color = new Color(0f, 0f, 1f);
else
    Gizmos.color = new Color(Mathf.Clamp01(1 - score), Mathf.Clamp01(score), 0);
Gizmos.DrawWireSphere(point.point.pos, 0.1f);
```

예약에서 점유로 전환

`QueryNewPoint`는 타깃, 전체 늑대 목록, 후보 지점으로 `EQSContext`를 구성한다. 선택한 지점의 NavMesh 구역이 몬스터의 구역과 같은지 확인하고 기존 점유-예약을 갱신한다. 새 지점은 `_goalPoint`로 저장하며 `MoveToGoal`이 `NavMeshAgent.SetDestination`에 전달한다.

늑대의 EQS 후보 평가와 위치 예약·계속

Assets/04.Scripts/CommonScripts/EQSSystem/EQSBehavior/EQSQuerierBehaviour_Battle.cs 발췌

CSHARP

```
private void ReservePoint(EQSPoint point)
{
    if(null!= _reservedPoint && _reservedPoint != point )
    {
        _reservedPoint.reserver = null;
    }
    point.reserver = _querier;
    _reservedPoint= point;
    _lastReservationTime =Time.time;
}
```

목표 지점까지의 거리가 `_stoppingDistance`보다 작아지면 `OccupyPoint`가 점유자를 등록한다. 이때 예약자와 `_goalPoint`를 비우고 `OccupyingPoint`에 도착 지점을 보관한다. 전투 이동 클래스에는 방향별 관심도·위험도 계산과 벽 주변 이동 처리도 함께 들어 있다.

Assets/04.Scripts/CommonScripts/EQSSystem/EQSBehavior/EQSQuerierBehaviour_Battle.cs 발췌

CSHARP

```
private void OccupyPoint(EQSPoint point)
{
    if (_querier == point.occupant) return;
    ReleaseOccupyingPoint();
    if (null!= _querier?.OccupyingPoint)
    {
        _querier.OccupyingPoint.occupant =null;
    }
    point.occupant= _querier;
    point.reserver = null;
    _querier.OccupyingPoint = point;
    _reservedPoint = null;
    _goalPoint = null;
}
```



플레이어 주변에서 서로 다른 위치를 차지하는 늑대들

05 / 데이터와 표현

CSV 관리와 대화 노드 연결

DialogueNodes와 DialogueEdges를 읽어 대사 목록과 선택지의 다음 노드 연결을 구성했다.

블로그 원문 ↗

대사와 연결을 두 표로 관리

대화 데이터는 `Assets/Resources/Data/CSV` 아래의 `DialogueNodes.csv`와 `DialogueEdges.csv`에 저장한다. Nodes는 대사와 연출을, Edges는 다음 노드와 선택지 정보를 담는다.

파일	주요 필드	사용하는 정보
DialogueNodes	EventName, SpeakerName, NodeID, IsBranch	대화 묶음·화자·노드·분기 여부
DialogueNodes	Animation, Script, DialogueEffect, EffectAmount	애니메이션·대사·효과·수치
DialogueEdges	EventName, FromNodeID, ToNodeID	대화 묶음 안에서의 노드 연결
DialogueEdges	ButtonLabel, Type	선택지 문구와 응답 타입

The image shows two side-by-side CSV files. The left file is 'DialogueNodes.csv' and the right file is 'DialogueEdges.csv'. Both files have columns labeled A through M. The 'DialogueNodes.csv' file contains dialogue events with details like speaker names, node IDs, and various effects. The 'DialogueEdges.csv' file contains information about the connections between different dialogue nodes, including from/to node IDs and button labels.

DialogueNodes의 대사 표와 DialogueEdges의 선택지 연결 표

디렉터리와 실행 흐름

TEXT

Assets/04.Scripts/
└ UI/Dialogue/Utility/DialogueParser.cs

CSV 관리와 대화 노드 연결 · 계속

TEXT

DialogueEdges → 이벤트·출발 노드별 연결 목록
DialogueNodes → 대사·연출 목록
→ NodeID로 결합 → 선택지와 다음 대사

TextAsset을 읽어 이벤트별로 구성

CSVDialogueParser는 두 파일을 Resources.Load<TextAsset>로 읽는다. 줄바꿈을 LF로 통일하고 빈 행을 제외한다. 첫 행은 헤더이므로 인덱스 1부터 해석한다. 행의 열은 쉼표로 나눈다.

Assets/04.Scripts/UI/Dialogue/Utility/DialogueParser.cs 발췌

CSHARP

```
TextAsset EdgeAsset = Resources.Load<TextAsset>("Data/CSV/" + _EdgeFileName);
TextAsset NodeAsset = Resources.Load<TextAsset>("Data/CSV/" + _NodeFileName);
Dictionary<string, Dialogues> DialogueDictionary = new Dictionary<string, Dialogues>();
if (NodeAsset == null || EdgeAsset == null)
{
    Debug.LogError("Dialogue CSV files not found.");
    return DialogueDictionary;
}
var Lines = EdgeAsset.text.Replace("\r\n", "\n").Replace("\r", "\n");
var EdgeRows = Lines.Split('\n', StringSplitOptions.RemoveEmptyEntries);
```

Edges는 대화 이벤트 이름 → 출발 NodeID → 도착 NodeID와 EdgeData 목록의 중첩 Dictionary로 구성한다. EdgeData에는 버튼 문구와 응답 타입을 보관한다. EventName이 비어 있는 다음 행들은 이전 이벤트 이름을 사용한다.

Assets/04.Scripts/UI/Dialogue/Utility/DialogueParser.cs 발췌

CSHARP

```
int ToNodeID = int.Parse(EdgeRowElements[(int)DIALOGUE_EDGE.TO_NODEID].Trim());
int FromNodeID = int.Parse(EdgeRowElements[(int)DIALOGUE_EDGE.FROM_NODEID].Trim());
EdgeData NowEdgeData = new EdgeData();
if(!EdgeDataDictionary[NowEventName].ContainsKey(FromNodeID))
    EdgeDataDictionary[NowEventName][FromNodeID] = new List<Tuple<int, EdgeData>>();
NowEdgeData.ButtonLabel = EdgeRowElements[(int)DIALOGUE_EDGE.BUTTON_LABEL].Trim();
NowEdgeData.myResponse = EdgeRowElements[(int)DIALOGUE_EDGE.MY_RESPONSE].Trim();
EdgeDataDictionary[NowEventName][FromNodeID].Add(new Tuple<int, EdgeData> (ToNodeID, NowEdgeData ));
```

연속 대사와 선택지 구성

Nodes에서 NodeID가 있는 행을 만나면 새 Dialogue를 만들고 NodeIDToIndex에 등록한다. 다음 NodeID가 나올 때까지 대사, 애니메이션, 효과 수치를 같은 객체의 목록에 넣는다. 대사 안의 작은따옴표는 쉼표로, &는 줄바꿈으로 바꾼다.

CSV 관리와 대화 노드 연결 · 계속

[Assets/04.Scripts/UI/Dialogue/Utility/DialogueParser.cs](#) 발췌

C#

```
NowDialogue.Scripts.Add(NodeElements[(int)DIALOGUE_NODE.SCRIPT].ToString().Replace("'", "").Replace("&", "\n"));
NowDialogue.AnimationName.Add(NodeElements[(int)DIALOGUE_NODE.ANIMATION_NAME].ToString().Trim());
{
    isBranch = NodeElements[(int)DIALOGUE_NODE.IS_BRANCH].ToString().Trim() == "TRUE";
}
```

마지막으로 해당 NodeID의 Edge 목록을 읽어 [NextBranchNodeID](#), [Labels](#), [ResponseTypes](#)를 채운다. 대화 UI는 이 연결 정보를 사용해 선택지 버튼과 다음 대사를 표시한다.

CSV 파일의 저장 인코딩은 UTF-8 BOM을 사용한다. 헤더를 건너뛰는 파서는 본문 행을 그대로 읽고, Excel에서는 UTF-8 텍스트로 표를 열 수 있다.

06 / UI와 상호작용

퀘스트·다이얼로그와 지역 방문 이벤트

대화와 지역 방문 이벤트를 목표 평가기에 전달하고 완료 보상과 다음 퀘스트 활성화를 처리했다.

[블로그 원문 ↗](#)

Quest And Dialogue · 대화, 보상 지급과 퀘스트 진행 · [YouTube에서 보기](#)

Patrol Event By Quest · 순찰 지역 방문과 목표 진행 · [YouTube에서 보기](#)

대화에서 퀘스트 진행으로 연결

다이얼로그는 CSV의 대사와 선택지 연결을 읽어 표시한다. 퀘스트 정의는 `QuestData`, 진행 상태는 `QuestRuntimeProcess`와 `SubTaskRuntimeProcess`로 관리한다. 퀘스트 안의 목표는 대화, 장착, 소비, 지역 방문, 스킬 학습, 처치, 수집, 강화 타입으로 구분한다.

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/  
├ Managers/QuestManager.cs  
├ Quest/TaskStorage/TaskEvaluator.cs  
├ Quest/TaskStorage/Task_TalkToNPC.cs  
└ Quest/TaskStorage/Task_VisitArea.cs
```

TEXT

```
퀘스트 수락 → 목표 이벤트 구독 → 진행량 갱신 → 다음 목표  
→ 전체 목표 완료 → 보상 지급 → 완료 이벤트 → 다음 퀘스트 조건 평가
```

퀘스트 수락과 목표 평가기

`QuestManager.ActivateQuest`는 선행 조건과 `QUEST_READY` 상태를 검사한다. 수락하면 상태를 `QUEST_ACCEPTED`로 바꾸고 목표 진행 객체를 구성한다. `ReconcileTaskStates`가 처음으로 끝나지 않은 목표를 활성화하고 평가기의 이벤트 구독을 시작한다.

대화 목표의 `Task_TalkToNPC`는 NPC ID, 퀘스트 코드, 목표 코드, 목표 타입과 수락 상태를 검사한다. 조건이 맞으면 진행량을 증가시킨다. 목표가 끝나면 구독을 해제하고 다음 목표를 활성화한다.

퀘스트·다이얼로그와 지역 방문 이벤트 · 계속

[Assets/04.Scripts/Quest/TaskStorage/TaskEvaluator.cs](#) 발췌

```
CSHARP
public void justCheckNowAndCheckNext()
{
    if (_subTaskRunTimeProcess._runTimeProcess.TaskState != eQuestTaskState.ACCEPTED) return;
    if (_goalAmount <= _subTaskRunTimeProcess._runTimeProcess.CurrentAmount)
    {
        _subTaskRunTimeProcess._runTimeProcess.TaskState = eQuestTaskState.COMPLETED;
        ReadyUnsubscribe();
        Managers.Quest.ActivateNextTask(_questtCode, _taskCode);
    }
    Managers.Event.Publish(new Event_TaskUpdated(_questtCode, _taskCode));
}
```

완료 처리와 보상 지급

`CheckAndCallCompleteEvent`는 모든 하위 목표의 완료 상태를 확인한다. 퀘스트를 완료 상태로 바꾸고 `QuestData.GiveReward`를 호출한 다음 `Event_QuestCompleted`를 발행한다. `GiveReward`는 아이템을 인벤토리에 추가하고 골드와 경험치를 플레이어에 전달한다.

[Assets/04.Scripts/Quest/TaskStorage/TaskEvaluator.cs](#) 발췌

```
CSHARP
public void CheckAndCallCompleteEvent()
{
    if (_questRunTimeProcess._runtimeProcess.State != QUEST_STATE.QUEST_ACCEPTED) return;
    List<SubTask> subTasks = _questData.tasks;
    foreach (SubTask subTask in subTasks)
    {
        if (Managers.Quest.GetTaskRunTimeProcess(_questtCode, subTask.SubTaskCODE)
            ._runTimeProcess.TaskState != eQuestTaskState.COMPLETED)
            return;
    }
    foreach (SubTask subTask in subTasks)
        Managers.Quest.GetTaskRunTimeProcess(_questtCode, subTask.SubTaskCODE)
            ._taskEvaluator.ReadyUnsubscribe();
    _questRunTimeProcess._runtimeProcess.State = QUEST_STATE.QUEST_COMPLETED;
    var completed = new Event_QuestCompleted(_questtCode);
    _questData.GiveReward(completed);
    Managers.Event.Publish(completed);
}
```

`QuestManager`는 퀘스트 완료와 레벨 상승 이벤트를 구독한다. 이벤트를 받으면 `UpdateReadyQuests`가 선행 조건을 다시 평가한다. 조건을 충족한 퀘스트는 준비 상태가 되고 `useAutoAcception`이 켜져 있으면 수락까지 진행한다.

순찰 퀘스트의 지역 방문

`Task_VisitArea`는 수신한 지역 ID와 목표 ID가 같고 목표가 수락 상태일 때 진행량을 증가시킨다. 이후 공통 평가기의 목표 완료와 퀘스트 완료 검사를 호출한다. 대화와 지역 방문은 서로 다른 사건을 받지만 같은 목표 진행 구조를 사용한다.

퀘스트·다이얼로그와 지역 방문 이벤트 · 계속

진행 상태별 퀘스트 목록

퀘스트 UI의 목록은 `GetReadyQuests`, `GetOnQuests`, `GetCompletedQuests`로 나눈다. 각 메서드는 런타임 객체의 상태를 기준으로 준비·진행·완료 목록을 반환한다.

`Assets/04.Scripts/Managers/QuestManager.cs` 발췌

C#

```
public List<QuestRuntimeProcess> GetReadyQuests()
{
    return _questRunTimeProcesses.Values.Where(x => x._runtimeProcess.State == QUEST_STATE.QUEST_READY).ToList();
}
public List<QuestRuntimeProcess> GetCompletedQuests()
{
    return _questRunTimeProcesses.Values.Where(x => x._runtimeProcess.State == QUEST_STATE.QUEST_COMPLETED).ToList();
}
public List<QuestRuntimeProcess> GetOnQuests()
{
    return _questRunTimeProcesses.Values.Where(x => x._runtimeProcess.State == QUEST_STATE.QUEST_ACCEPTED).ToList();
}
```



퀘스트 UI의 수락 가능한 목록

퀘스트·다이얼로그와 지역 방문 이벤트 · 계속



퀘스트 11의 진행 중인 목표 목록



퀘스트 UI의 완료 목록

퀘스트·다이얼로그와 지역 방문 이벤트 · 계속



NPC 대화와 퀘스트 진행을 연결한 플레이 화면

07 / UI와 상호작용

상점 목록과 아이템 구매·판매

ShopInfo의 판매 목록을 아이템 정의와 연결하고 거래 확인 UI에서 수량과 금액을 처리했다.

[블로그 원문 ↗](#)

판매 목록과 아이템 정의 연결

ShopManager는 상인별 ShopInfo를 보관한다. 첫 상점의 파일 경로는 `Application.persistentDataPath + "/ShopInfo.json"`이다. LoadInfo는 JSON을 ShopInfo로 역직렬화한다.

판매 목록의 ID는 DataManager.GetItemData로 아이템 정의와 연결한다. ShopGridScrollView가 선택한 종류에 맞는 셀을 만들고 아이콘과 아이템 정보를 표시한다. 상점의 장비·소비 아이템 버튼은 Refresh에 다른 아이템 타입을 전달한다.



상점에서 룬 소드의 상세 정보와 가격을 표시한 화면

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/
├ Managers/Shop/ShopManager.cs
├ UI/Shop/ShopDirector.cs
├ UI/Shop/ShopGridScrollView.cs
└ UI/ConfirmMessage/ConfirmTrade.cs
```

TEXT

```
ShopInfo → 아이템 ID 조회 → 상품 셀 → 상품 선택
→ 거래 수량 확인 → 인벤토리 변경 → 소지금 변경
```

상점 목록과 아이템 구매·판매 · 계속

상품 선택과 거래 확인

ShopDirector는 선택한 셀의 ItemID와 ItemInfo를 ConfirmTrade에 전달한다. 구매는 POLICY_BUY, 수량을 입력하는 판매는 POLICY_SELL로 처리한다. 아이템 선택 시에는 상세 팝업을 열어 정의 데이터를 표시한다.

Assets/04.Scripts/UI/Shop/ShopDirector.cs 발췌

C#

```
_btnBuy.onClick.AddListener(() =>
{
    ShopGridCellView nowCellView = m_ShopGridScrollView.CurrentCellView;
    if (null == nowCellView) { _textAdvice.text="구매할 상품을 선택해주세요."; return; }
    int itemID = nowCellView.ItemID;
    _confirmTrade.OnOpen().Refresh(itemID,nowCellView.ItemInfo ,eTRADEPOLICY.POLICY_BUY);
});
```

ConfirmTrade는 입력 문자열을 정수로 변환한다. 구매 수량이 양수이고 보유 금액이 $\text{수량} \times \text{가격}$ 이상이면 _allowTobuy를 설정한다. 거래 버튼은 이 플래그를 확인한 뒤 인벤토리에 아이템을 추가하고 금액을 차감한다.

Assets/04.Scripts/UI/ConfirmMessage/ConfirmTrade.cs 발췌

C#

```
case eTRADEPOLICY.POLICY_BUY:
    if (false == _allowTobuy) return;
    Managers.Inventory.TryAddItem(itemType, _itemID, _resultInt);
    Managers.Player.AddMoney(-money);
    break;
```

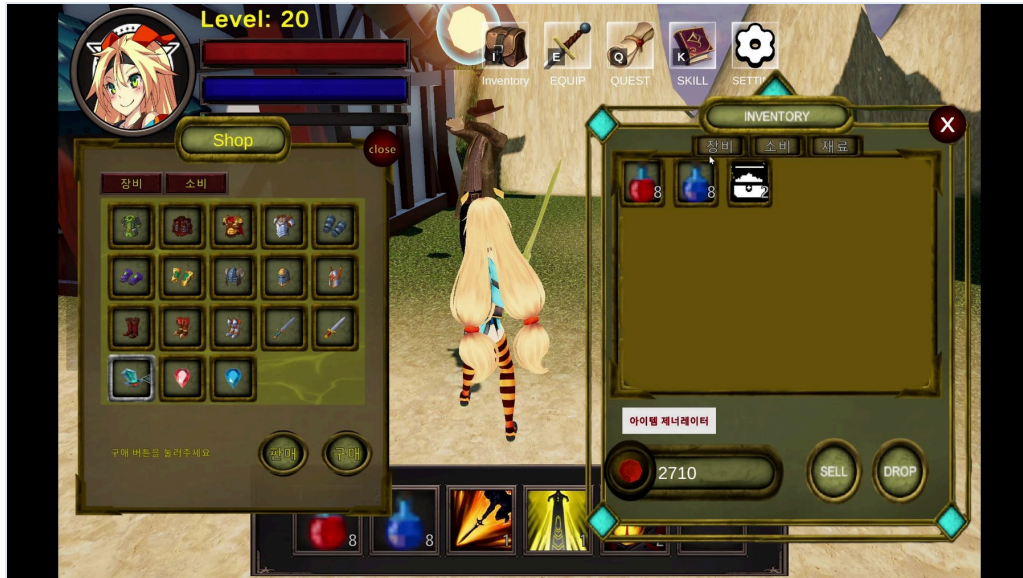


거래 확인 팝업에서 구매 수량을 입력하는 장면

상점 목록과 아이템 구매·판매 · 계속

판매와 인벤토리 반영

장비 판매는 선택한 `ItemInfo` 한 개를 제거하고 정의 데이터의 가격을 더한다. 소비 아이템 등 수량형 거래는 확인 창 입력 수량을 사용한다. 상품 정의는 ID로 조회하고, 소유한 장비 객체는 `ItemInfo`로 구분한다.



상점 거래 후 인벤토리에 표시한 아이템

08 / UI와 상호작용

무기 강화 수치와 속성 이펙트

장비 인스턴스의 추가 능력치를 계산하고 강화 이벤트와 단계별 파티클을 연결했다.

[블로그 원문 ↗](#)

강화 대상은 장비 인스턴스

`UI_ReinforceDirector.OnDrop`은 드래그한 아이콘의 데이터를 풀에서 빌린 아이콘에 복사한다. 아이템 타입이 장비인지 확인하고 강화 슬롯에 배치한다. `_nowIDs`가 선택한 장비 데이터를 보관하며, 강화 버튼은 `InstanceItemInfo`로 변환해 개별 장비의 강화 수치를 변경한다.



무기 강화 슬롯과 능력치 증가 결과

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/
├─ UI/Reinforce/UI_ReinforceDirector.cs
├─ Effects/ReinforceEffect/ReinforceEffect.cs
└─ Managers/WeaponEffectManager/WeaponEffectManager.cs
```

TEXT

```
장비 드롭 → InstanceItemInfo → 강화 수치 계산
  → Event_Reinforce → 결과 표시
속성·강화 단계 → ReinforceEffect → 풀의 파티클 → 무기 MeshRenderer
```

무기 강화 수치와 속성 이펙트 · 계속

비용과 증가량 계산

강화 버튼은 연출 코루틴이 실행 중이거나 최대 강화 횟수에 도달한 경우 반환한다. 비용은 `instance.Level × 60`이며 증가량의 기준은 장비 레벨에 1~3의 정수 난수를 곱해 정한다.

`Assets/04.Scripts/UI/Reinforce/UI_ReinforceDirector.cs` 발췌

CSHARP

```
int reinforceIntensity = instance.Level;
int randomVar = UnityEngine.Random.Range(1, 4);
reinforceIntensity *= randomVar;
```

장비 부위별로 `ReinforceResult`의 능력치를 채운다. 무기는 기준 증가량을 공격력·치명타 확률·치명타 피해에 더하고, 방어력에는 2배, 최대 체력과 마나에는 5배를 사용한다.

`Assets/04.Scripts/UI/Reinforce/UI_ReinforceDirector.cs` 발췌

CSHARP

```
case eEQUIPMENTTYPE.WEAPON:
    result.Defense +=reinforceIntensity*2;
    result.Damage +=reinforceIntensity;
    result.CriChance +=reinforceIntensity;
    result.CriDamage +=reinforceIntensity;
    result.MaxHP +=reinforceIntensity*5;
    result.MaxMP +=reinforceIntensity*5;
    break;
```

0~8의 정수 난수가 6 이하일 때 계산한 추가 능력치를 장비 인스턴스에 반영한다. 그 외에는 빈 결과를 사용한다.

`CurrentReinforce`는 시도 후 증가하며, `Event_Reinforce`를 발행하고 결과 표시기를 갱신한다.

`Assets/04.Scripts/UI/Reinforce/UI_ReinforceDirector.cs` 발췌

CSHARP

```
if(UnityEngine.Random.Range(0,9)<=6)
{
    instance.ExtraCriChance +=result.CriChance;
    instance.ExtraCriDamage +=result.CriDamage;
    instance.ExtraDamage +=result.Damage;
    instance.ExtraDefense +=result.Defense;
    instance.ExtraHealth +=result.MaxHP;
    instance.ExtraMana +=result.MaxMP;
    result.bReinforceResult = true;
}
else
{
    result = new ReinforceResult();
    ++instance.CurrentReinforce;
    Managers.Event.Publish<Event_Reinforce>(new Event_Reinforce(itemInfo));
    _resultDisplayer.SetResult(result);
}
```

무기 강화 수치와 속성 이펙트 · 계속



강화한 무기의 아이콘과 강화 결과 표시

강화 단계에 맞는 파티클 선택

`ReinforceEffect` SO는 속성과 `ReinforceInfo` 목록을 가진다. 각 항목은 적용 단계와 이펙트 프리팹을 저장한다.

`WeaponEffectManager`는 속성별 SO를 읽어 프리팹 풀을 준비한다.

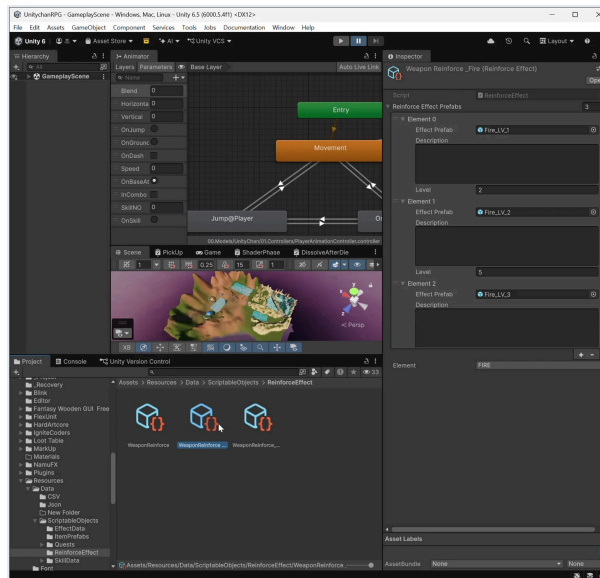
`ActivateEffect`는 목록을 앞에서부터 순회하면서 장비 강화 단계 이하인 마지막 항목을 선택한다. 이 순회는 단계가 오름차순인 목록을 사용한다. 선택한 프리팹을 풀에서 가져오고 장비 인스턴스 ID로 핸들을 보관한다.

`Assets/04.Scripts/Managers/WeaponEffectManager/WeaponEffectManager.cs` 발췌

CSHARP

```
public void ActivateEffect(ELEMENT element,int level,string instanceID ,MeshRenderer mr ,Transform parent)
{
    var effects = _dicEffect[element].ReinforceEffectPrefabs;
    _nowEffect =null;
    foreach (var effect in effects) {
        if (effect == null) continue;
        if (effect.Level > level) break;
        _nowEffect=effect;
    }
    if (null ==_nowEffect) return;
    Poolable handle= Managers.Pool.LendPoolableTo(_nowEffect.EffectPrefab.name,null);
    SetWeaponMesh(handle,mr);
    handle.transform.SetParent(parent);
    _reinforceEffects.Add(instanceID, handle);
}
```

무기 강화 수치와 속성 이펙트 · 계속



강화 단계별 이펙트 프리팹을 지정한 SO Inspector

`SetWeaponMesh`는 파티클 Shape을 `MeshRenderer`로 설정하고 무기의 렌더러를 연결한다. 무기 메시를 파티클 발생 영역으로 사용하며, 비활성화할 때는 메시 참조를 비우고 이펙트를 풀에 반환한다.

`Assets/04.Scripts/Managers/WeaponEffectManager/WeaponEffectManager.cs` 발췌

CSHARP

```
public void SetWeaponMesh(Poolable handle, MeshRenderer meshRenderer)
{
    var particleSystems = handle.GetComponentsInChildren<ParticleSystem>(true);
    foreach (var particleSystem in particleSystems)
    {
        var shape = particleSystem.shape;
        shape.enabled = true;
        shape.shapeType = ParticleSystemShapeType.MeshRenderer;
        shape.meshRenderer = meshRenderer;
        shape.skinnedMeshRenderer = null;
    }
}
```

무기 강화 수치와 속성 이펙트 · 계속



강화된 무기를 장착한 상태의 장비·능력치 화면

09 / 데이터와 표현

ScriptableObject로 스킬·퀘스트·이펙트 관리

SO의 직렬화 필드와 에셋 목록을 정의하고 스킬 이벤트, 퀘스트 카탈로그, 강화 이펙트에서 읽어 사용했다.

[블로그 원문 ↗](#)

SO 에셋과 실행 코드 연결

SO 에셋은 `Assets/Resources/Data/ScriptableObjects`에 둔다. C# 클래스에 `CreateAssetMenu`를 선언해 에디터에서 생성하고 `SerializeField`로 Inspector에 설정값을 노출한다. 스킬은 수치와 애니메이션 이벤트, 퀘스트는 목표와 보상, 강화 효과는 속성과 단계별 프리팹을 저장한다.

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/  
├─ Quest/ScriptableObject/QuestData.cs  
├─ Quest/QuestCatalog/QuestCatalog.cs  
└─ Effects/ReinforceEffect/ReinforceEffect.cs
```

```
Assets/Resources/Data/ScriptableObjects/SkillData/  
├─ SkillSO.cs  
└─ SkillSOEditor.cs
```

TEXT

SO 클래스의 직렬화 필드 → Inspector에서 에셋 설정
→ Resources 조회 또는 SO 참조 전달 → 실행 코드에서 필드 사용

SkillSO의 직렬화 필드

스킬 실행 코드와 UI는 SO의 프로퍼티로 수치와 아이콘을 읽는다. `handle`은 쿼 슬롯에 연결하는 정수 식별자다.

ScriptableObject로 스킬-퀘스트-이펙트 관리 · 계속

Assets/Resources/Data/ScriptableObjects/SkillData/SkillSO.cs 발췌

CSHARP

```
[System.Serializable, CreateAssetMenu(fileName = "SkillSO", menuName = "Scriptable Objects/SkillSO")]
public class SkillSO : ScriptableObject
{
    [SerializeField] EventSequence stateNodeEvents;
    [SerializeField] Sprite skillIcon;
    [SerializeField, TextArea(2, 5)] string skillName;
    [SerializeField, TextArea(2, 5)] string skillDesc;
    [SerializeField, TextArea(2, 5)] string skillClass;
    [SerializeField] float coolTime;
    [SerializeField] float manaCost;
    [SerializeField] float range;
    [SerializeField] float damage;
    [SerializeField] float damageIncreasePercentPoint;
    [Header("Buff")]
    [SerializeField] BuffInfo[] buffInfo;
    [SerializeField] int skillNO;
    [SerializeField] int skillLevel;
    [SerializeField] int openLevel;
    [SerializeField] eSkillType skillType;
    [Header("Handle")]
    [SerializeField] public int handle;
    [SerializeField] public string alias;
    public EventSequence EventSequence => stateNodeEvents;
    public Sprite SkillIcon => skillIcon;
```

CSHARP · 계속

```
    public string SkillName => skillName;
    public string SkillDesc => skillDesc;
    public string SkillClass => skillClass;
    public float CoolTime => coolTime;
    public float ManaCost => manaCost;
    public float Range => range;
    public float Damage => damage;
    public int SkillNO => skillNO;
    public int OpenLevel => openLevel;
    public float DamageIncreasePercentPoint { get=> damageIncreasePercentPoint; set=> damageIncreasePercentPoint = value; }
    public eSkillType SkillType { get => skillType; }
    public BuffInfo[] BuffInfo { get => buffInfo; set => buffInfo = value; }
    public int SkillLevel { get=> skillLevel; set=> skillLevel =value; }
}
```

SkillSO의 이벤트 구조

SkillSO는 아이콘, 이름, 설명, 쿨타임, 마나 비용, 사거리, 피해량, 스킬 종류와 핸들을 보관한다. 애니메이션 설정은 `EventSequence` → `EventClipInfo` → `AnimEventDesc`로 구성한다. 각 클립에 애니메이션 전체 경로와 타격 부위를 지정하고 이벤트 목록을 연결한다.

ScriptableObject로 스킬·퀘스트·이펙트 관리 · 계속

Assets/Resources/Data/ScriptableObjects/SkillData/SkillISO.cs 발췌

CSHARP

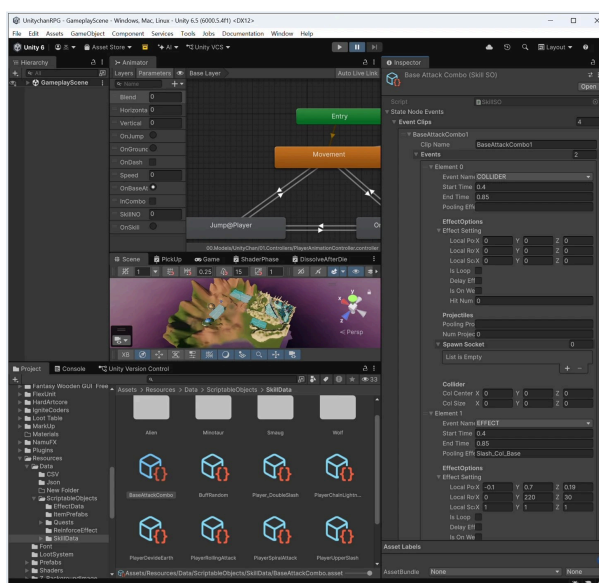
```
[System.Serializable]public struct EventClipInfo
{
    [SerializeField] public string clipName;
    [SerializeField] public List<AnimEventDesc> events;
    [SerializeField] public string animFullPath;
    [SerializeField] public eHitAreaMarker hitArea;
}

[System.Serializable]public struct EventSequence
{
    [SerializeField] public List<EventClipInfo> eventClips;
```

Assets/Resources/Data/ScriptableObjects/SkillData/SkillISO.cs 발췌

CSHARP

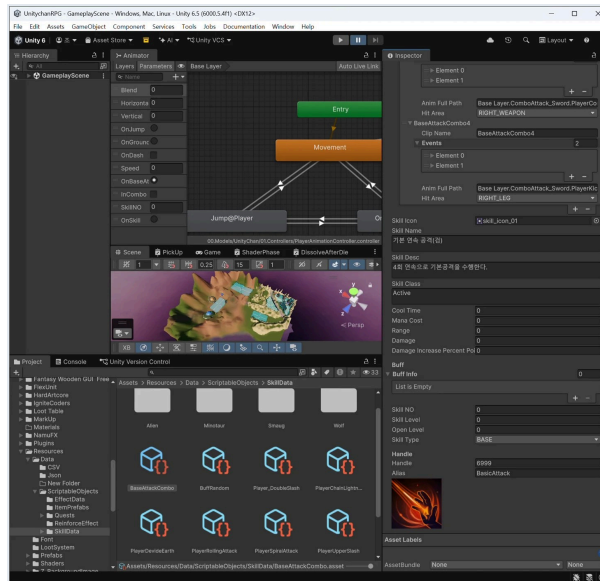
```
[System.Serializable]public struct AnimEventDesc
{
    [SerializeField] public eAnimEvent eventName;
    [SerializeField] public float startTime;
    [SerializeField] public float endTime;
    [SerializeField] public string poolingEffectName;
    [Header("EffectOptions")]
    [SerializeField] public EffectSetting effectSetting;
    [Header("Projectiles")]
    [SerializeField] public string poolingProjectileKey;
    [SerializeField] public int numProjectiles;
    [SerializeField] public string[] spawnSocket;
    [Header("Collider")]
    [SerializeField] public Vector3 colCenter;
    [SerializeField] public Vector3 colSize;
}
```



SkillISO의 클립별 이벤트 목록과 시간 설정

ScriptableObject로 스킬-퀘스트-이펙트 관리 · 계속

AnimEventDesc는 이벤트 종류와 시작-종료 시간, 폴링 키, 소켓, 충돌체 범위를 가진다. **EffectSetting**에는 로컬 위치-회전-스케일과 무기 부착 여부 등의 옵션을 저장한다.



SkillSO의 아이콘·이름·수치·종류를 설정하는 Inspector

애니메이션 시간으로 이벤트 실행

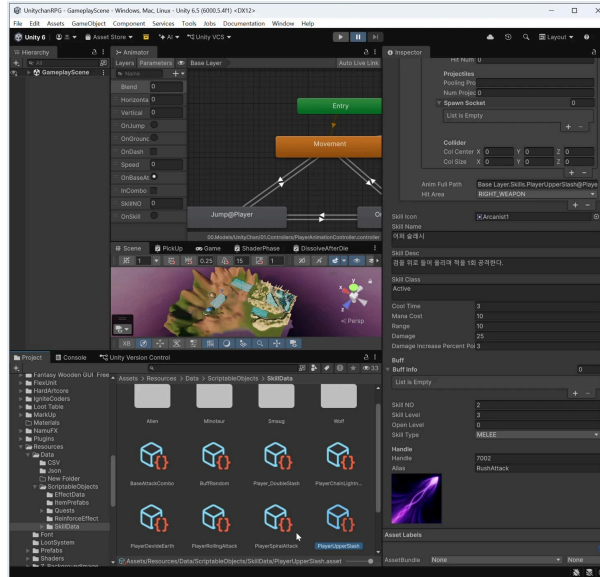
PlayerMeleeSkillState는 SO의 **animFullPath**를 Animator 해시로 변환한다. 해당 애니메이션에 진입하면 **normalizedTime**과 이벤트의 시작-종료 시간을 비교한다. 이전 프레임의 활성 여부와 달라진 이벤트만 **FireEvents**로 ON 또는 OFF를 전달한다.

Assets/04.Scripts/PlayerScripts/PlayerState/PlayerMeleeSkillState.cs 발췌

CSHARP

```
for (int i = 0; i < events.Count; ++i)
{
    nowEventOn = IsPointInRange(tNow, events[i].startTime, events[i].endTime);
    if (nowEventOn != isPrevEventOns[i])
    {
        if (nowEventOn)
            FireEvents(events[i].eventName, true, events[i]);
        else
            FireEvents(events[i].eventName, false, events[i]);
    }
    isPrevEventOns[i] = nowEventOn;
}
```

ScriptableObject로 스킬-퀘스트-이펙트 관리 · 계속



어퍼 슬래시의 애니메이션 이벤트와 효과 설정

`COLLIDER`, `EFFECT`, `SUMMON_PROJECTILES` 같은 종류별로 실행 코드가 나뉜다. 스킬 데이터에 저장한 시간과 소켓 정보를 읽어 히트박스, 파티클, 투사체를 제어한다.

QuestData와 QuestCatalog

퀘스트 정의 클래스에는 목표 목록, 보상, 선행 조건과 자동 수락 설정을 함께 둔다.

`Assets/04.Scripts/Quest/ScriptableObject/QuestData.cs` 발췌

CSHARP

```
[CreateAssetMenu(fileName = "QuestData", menuName = "Scriptable Objects/QuestData")]
public class QuestData : ScriptableObject
{
    [Header("Quest Description")]
    [SerializeField] public string QuestTitle;
    [TextArea(5,5)]
    [SerializeField] public string QuestDescription;
    [Header("Contents")]
    [SerializeField] public List<SubTask> tasks ;
    [Header("Reward")]
    [SerializeField] public QuestRewards QuestReward;
    [Header("Condition")]
    [SerializeField] public QuestPrerequisite AcceptanceCondition;
    [Header("NPC ")]
    [SerializeField] public string QuestCODE;
    [SerializeField] public int NPC_ID;
    public bool useAutoComplete = false;
    public bool useAutoAcceptance = false;
    public bool CheckPreRequisites()
    {
        Debug.Assert(null !=AcceptanceCondition);
        return AcceptanceCondition.CheckCondition();
    }
    public void GiveReward(Event_QuestCompleted evt)
```

ScriptableObject로 스킬·퀘스트·이펙트 관리 · 계속

CSHARP · 계속

```

{
    int gold = QuestReward.Gold;
    int exp = QuestReward.Exp;
    foreach(var Reward in QuestReward.RewardItemSets)
    {
        eITEMTYPE type = Managers.Data.GetItemData(Reward.ItemID).Type;
        Managers.Inventory.TryAddItem(type, Reward.ItemID, Reward.ItemAmount);
    }
    Managers.Player.AddMoney(gold);
    if (exp > 0) Managers.Player.AddEXP(exp);
}
}

```

QuestData는 퀘스트 제목·목표·보상·수락 조건을 정의한다. 목표 목록의 원소인 SubTask에는 식별 코드, 대상 ID, 요구량, 대화 키와 목표 타입이 들어간다.

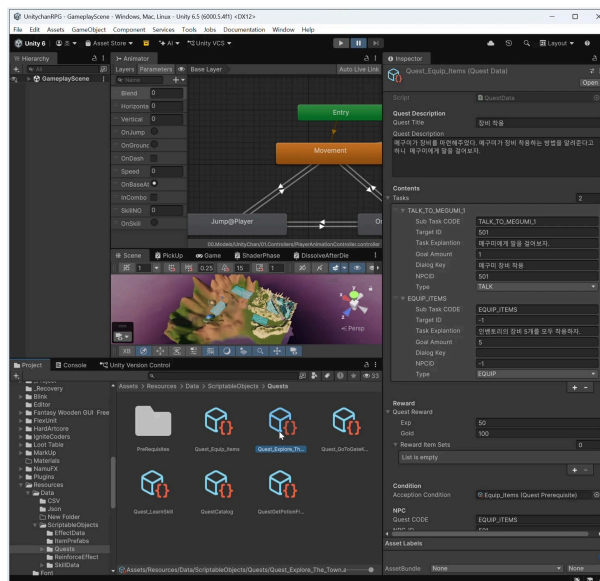
Assets/04.Scripts/Quest/ScriptableObject/QuestData.cs 발췌

CSHARP

```

[System.Serializable] public class SubTask
{
    [SerializeField] public string      SubTaskCODE;
    [SerializeField] public int         TargetID;
    [SerializeField] public string      TaskExplantion;
    [SerializeField] public int         GoalAmount;
    [SerializeField] public string      DialogKey;
    [SerializeField] public int         NPCID;
    [SerializeField] public eTASK_TYPE Type;
}

```



퀘스트 목표와 보상·수락 조건을 설정한 QuestData

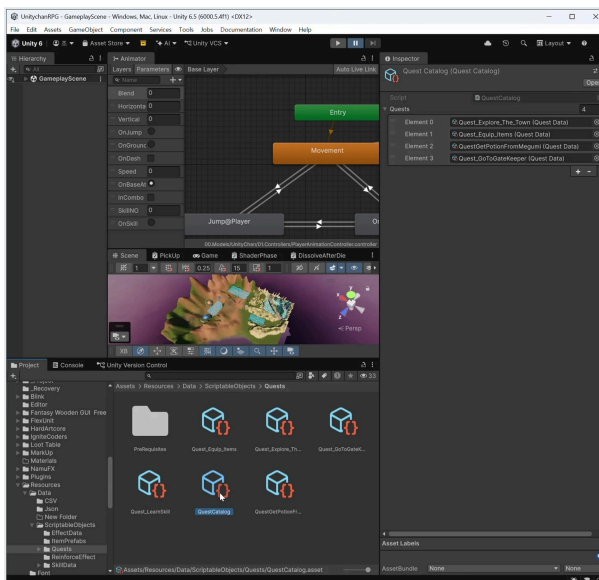
QuestCatalog는 여러 QuestData 에셋을 목록으로 묶는다. QuestManager.Init는 Resources.LoadAll<QuestCatalog>로 카탈로그를 읽고 SelectMany와 Distinct로 퀘스트를 모은 뒤 QuestCODE를 키로 등록한다. 진행량과 완료 상태는 런타임 DTO에 저장한다.

ScriptableObject로 스킬·퀘스트·이펙트 관리 · 계속

Assets/04.Scripts/Quest/QuestCatalog/QuestCatalog.cs 발췌

CSHARP

```
[CreateAssetMenu(fileName = "QuestCatalog", menuName = "Scriptable Objects/QuestCatalog")]
public class QuestCatalog:ScriptableObject
{
    [SerializeField]public List<QuestData> _quests;
}
```



QuestCatalog에 퀘스트 에셋들을 등록한 목록

속성별 강화 이펙트 SO

ReinforceEffect는 BASE, FIRE, ICE 속성 중 하나와 단계별 프리팹 목록을 갖는다. Inspector에서 프리팹과 적용 레벨을 지정하면 WeaponEffectManager가 속성과 강화 단계로 사용할 이펙트를 선택한다.

Assets/04.Scripts/Effects/ReinforceEffect/ReinforceEffect.cs 발췌

CSHARP

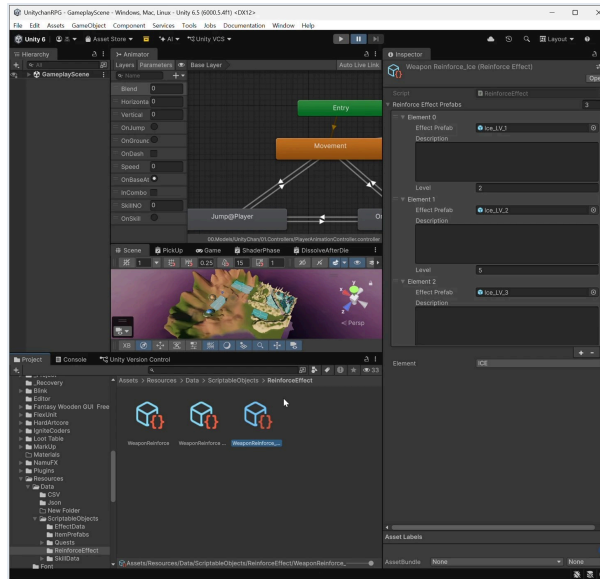
```
[CreateAssetMenu(fileName = "ReinforceEffect", menuName = "Scriptable Objects/ReinforceEffect")]
public class ReinforceEffect : ScriptableObject
{
    [SerializeField] public List<ReinforceInfo> ReinforceEffectPrefabs;
    [SerializeField] public ELEMENT Element;
}
```

Assets/04.Scripts/Effects/ReinforceEffect/ReinforceEffect.cs 발췌

CSHARP

```
[System.Serializable] public class ReinforceInfo
{
    [SerializeField] public GameObject EffectPrefab;
    [SerializeField, TextArea(5, 1)] public string Description;
    [SerializeField] public int Level;
}
```

ScriptableObject로 스킬-퀘스트-이펙트 관리 · 계속



속성별 강화 이펙트의 프리팹 참조와 레벨 설정

스킬 아이콘 미리보기

SkillSOEditor.cs의 ItemSOEditor는 CustomEditor(typeof(SkillISO))로 연결한다. 기본 Inspector를 그린 뒤 AssetPreview.GetAssetPreview로 스킬 아이콘을 가져와 100×100 영역에 표시한다. 변경이 생기면 EditorUtility.SetDirty(skillISO)를 호출한다.

10 / 데이터와 표현

JSON 정의 데이터와 플레이 진행 저장

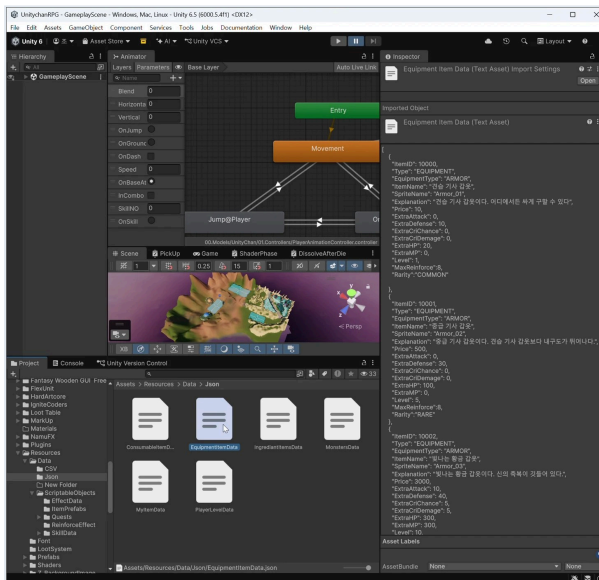
Resources의 JSON 정의와 persistentDataPath의 진행 정보를 구분하고 ID로 연결했다.

[블로그 원문 ↗](#)

JSON 파일의 두 용도

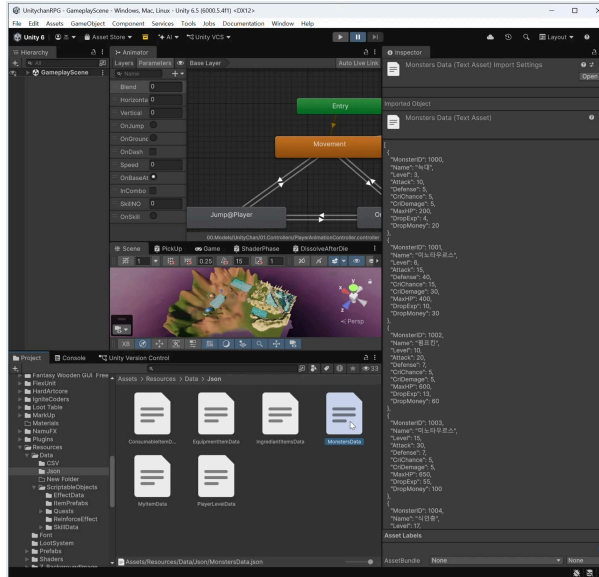
아이템·몬스터·레벨처럼 공통으로 읽는 정의 데이터는 `Resources/Data/Json`에 둔다. 플레이 중 바뀌는 인벤토리와 플레이어 정보, 퀘스트 진행은 `Application.persistentDataPath`의 JSON으로 저장한다. 상점 판매 목록인 `ShopInfo.json`도 이 경로에서 읽는다.

데이터	저장 형태	사용하는 내용
EquipmentItemData-ConsumableItemData-In gradientItemsData	Resources의 JSON	ID별 가격·아이콘·아이템 정의
MonstersData-PlayerLevel 계열 데이터	Resources의 JSON	몬스터와 성장에 쓰는 정의 값
QuestProcess.json	persistentDataPath의 JSON	퀘스트 코드와 진행 상태
QuestTaskProcess.json	persistentDataPath의 JSON	목표 코드·진행량·상태
ShopInfo.json	persistentDataPath의 JSON	상점에서 판매할 아이템 목록



JSON으로 관리하는 장비 아이템 정의

JSON 정의 데이터와 플레이 진행 저장 · 계속



JSON으로 관리하는 몬스터 정의 데이터

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/
├ Managers/DataManager.cs
├ Managers/QuestManager.cs
└ Managers/Shop/ShopManager.cs
```

TEXT

Resources JSON → 역직렬화 → ID Dictionary → 데이터 조회
 진행 DTO → 직렬화 → persistentDataPath JSON
 저장 JSON → 역직렬화 → 퀘스트·목표 코드로 런타임 결합

아이템 ID로 조회하는 Dictionary

`DataManager.LoadItemData`는 소비 아이템 JSON을 읽어 `ItemData[]`로 역직렬화한다. 이 배열을 `ItemID`를 키로 한 Dictionary로 만들고, 재료와 장비 데이터를 같은 Dictionary에 추가한다.

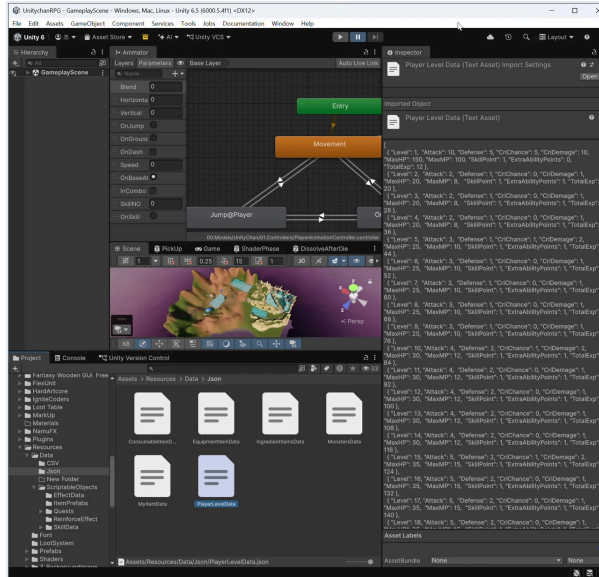
`Assets/04.Scripts/Managers/DataManager.cs` 발췌

CSHARP

```
TextAsset asset = Resources.Load<TextAsset>("Data/Json/ConsumableItemData");
var json = asset.text;
Debug.Log(json);
ItemData[] arrItemData = JsonConvert.DeserializeObject<ItemData[]>(json);
this.m_ItemData = arrItemData.ToDictionary(x => x.ItemID);
```

상점과 인벤토리는 `GetItemData(ItemID)`로 이 Dictionary를 조회한다. 소유 수량과 강화 수치 등 개별 상태는 아이템 정의와 연결된 소유 정보에서 관리한다.

JSON 정의 데이터와 플레이 진행 저장 · 계속



플레이어 레벨별 성장 정보를 보관한 JSON

퀘스트와 하위 목표 저장

`SaveQuests`는 퀘스트 런타임 객체의 DTO들을 목록으로 모아 직렬화한다. `SaveTasks`는 퀘스트별 목표 Dictionary를 순회하면서 하위 목표 DTO를 한 목록으로 저장한다.

`Assets/04.Scripts/Managers/QuestManager.cs` 발췌

C#

```
public void SaveQuests()
{
    var all = new List<QuestProcessDTO>();
    foreach (var pair in _questRunTimeProcesses)
    {
        all.Add(pair.Value._runtimeProcess);
    }
    var json = JsonConvert.SerializeObject(all, Formatting.Indented);
    File.WriteAllText(_pathQuest, json);
}
```

로드할 때는 먼저 카탈로그에서 퀘스트 정의를 등록한다. `LoadQuests`가 저장한 퀘스트 상태를 읽고 `LoadAndJoinTasks`가 `QuestCODE`와 `SubTaskCODE`로 목표 진행을 연결한다. `ReconcileTaskStates`는 완료 목표와 활성 목표를 정리하고 진행 중인 목표의 이벤트 구독을 구성한다.

11 / 전투와 행동

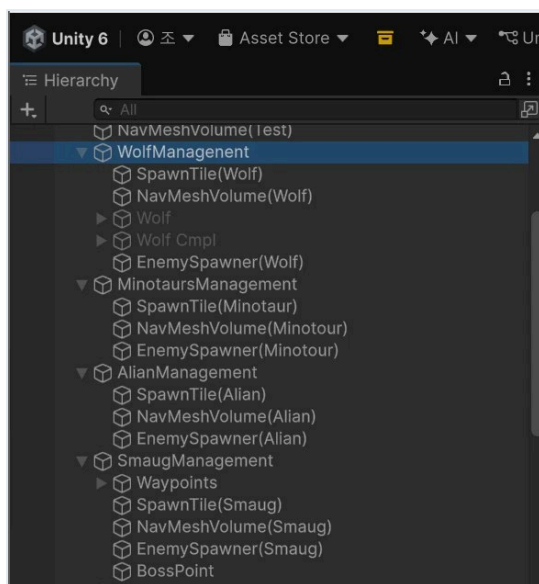
스포너 셀 배치와 박스 Gizmo

NavMesh 영역을 셀로 나누고 지면·이동 가능 위치를 검사한 뒤 몬스터와 디버그 박스를 배치했다.

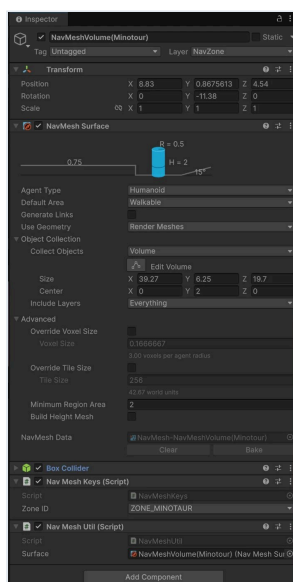
[블로그 원문 ↗](#)

몬스터별 스폰 영역 구성

썬은 몬스터 종류별 관리 오브젝트 아래에 **SpawnTile**, **NavMeshVolume**, **EnemySpawner**를 둔다. **NavMeshSurface**의 Volume 크기와 방향을 기준으로 셀 후보를 만든다. 관리 Inspector에는 NavMeshSurface, 구역 ID, NavMesh 유틸리티를 연결한다.



몬스터별 관리 오브젝트와 스폰·NavMesh 하위 구성



NavMesh Volume 크기와 미노타우루스 구역 ID 설정

스포너 셀 배치와 박스 Gizmo · 계속

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/  
├─ Monsters/Enemies/EnemySpawner.cs  
└─ Monsters/Enemies/SpawnTile.cs
```

TEXT

NavMesh Volume → 셀 후보 → 지면 Raycast → NavMesh 샘플 검사
→ 셀 등록 → 몬스터 배치·정보 주입 → 박스 Gizmo 표시

Volume 안에 셀 배치

`EnemySpawner.SpawnMonsterCells`는 Volume의 X-Z 크기를 셀 크기로 나눠 행과 열을 계산한다. 표면의 회전과 좌상단 기준 위치를 `SpawnTile.TestValidityAndSet`에 넘긴다. 셀은 인덱스를 행·열 좌표로 바꾸어 중심 위치를 계산한다.

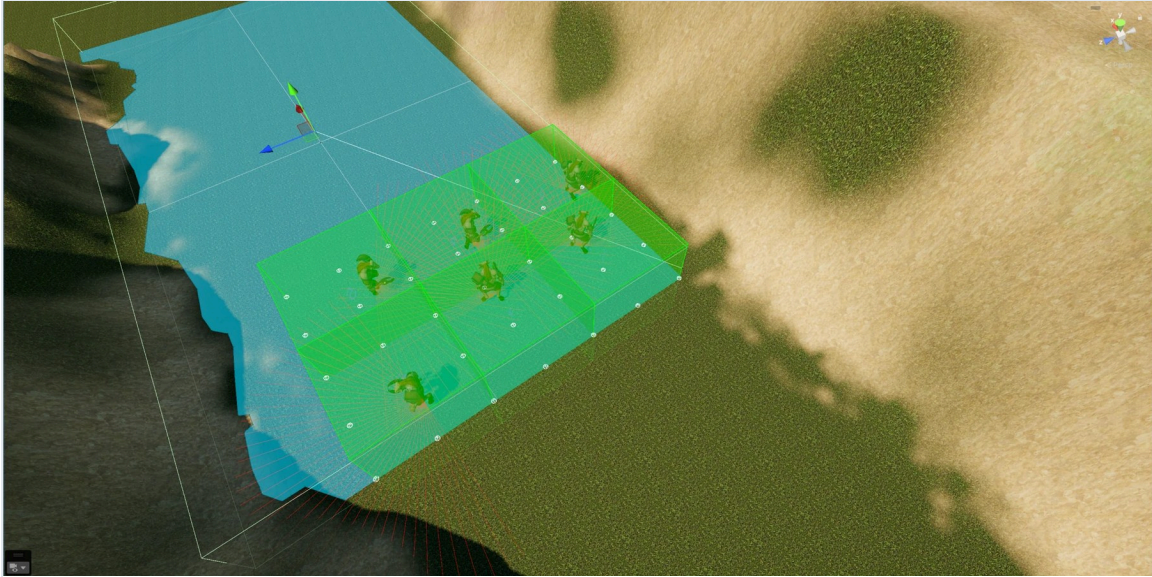
`Assets/04.Scripts/Monsters/Enemies/SpawnTile.cs` 발췌

CSHARP

```
int row = (int)volumeSize.x / (int)_cellWidth;  
int col = (int)volumeSize.y / (int)_cellDepth;  
_cellIndex = nowIndex;  
int idxX = _cellIndex/col;  
int idxZ = _cellIndex%col;  
_cellPosition =  
    posLT  
    +_cellWidth*(0.5f + idxX)* surface.transform.right.normalized  
    +_cellDepth*(0.5f + idxZ)* surface.transform.forward.normalized;  
_cellPosition.y+=_cellHeight;  
transform.position= _cellPosition;
```

셀 위에서 아래로 Raycast를 쏘아 Terrain을 찾는다. 지면 근처 NavMesh 위치를 샘플링하고, 셀 안의 3×3 후보점도 검사한다. 후보 9개가 모두 확보되면 `_isCellFixed`를 켜고 스폰에 사용할 셀로 등록한다.

스포너 셀 배치와 박스 Gizmo · 계속



NavMesh 위에 배치한 스폰 셀과 몬스터 주변 디버그 선

몬스터에 스폰너와 셀 인덱스 전달

`SetMonster`는 생성한 몬스터를 셀 위치에 두고 BehaviorGraphAgent의 Blackboard에 스폰너, 셀 인덱스, Animator와 스킬 번호를 전달한다. `IIndexConsumer`, `ISpawnerConsumer` 구현 객체에도 같은 정보를 주입한다. 마지막으로 `NavMeshAgent.Warp`로 에이전트 위치를 맞춘다.

셀 상태를 네모 박스로 표시

`OnDrawGizmos`는 `Gizmos.matrix`를 셀의 `localToWorldMatrix`로 바꾸어 회전과 위치를 반영한다. 확정된 셀은 초록색, 나머지는 보라색으로 표시한다. 반투명 큐브와 WireCube를 그린 뒤 이전 행렬을 복원한다.

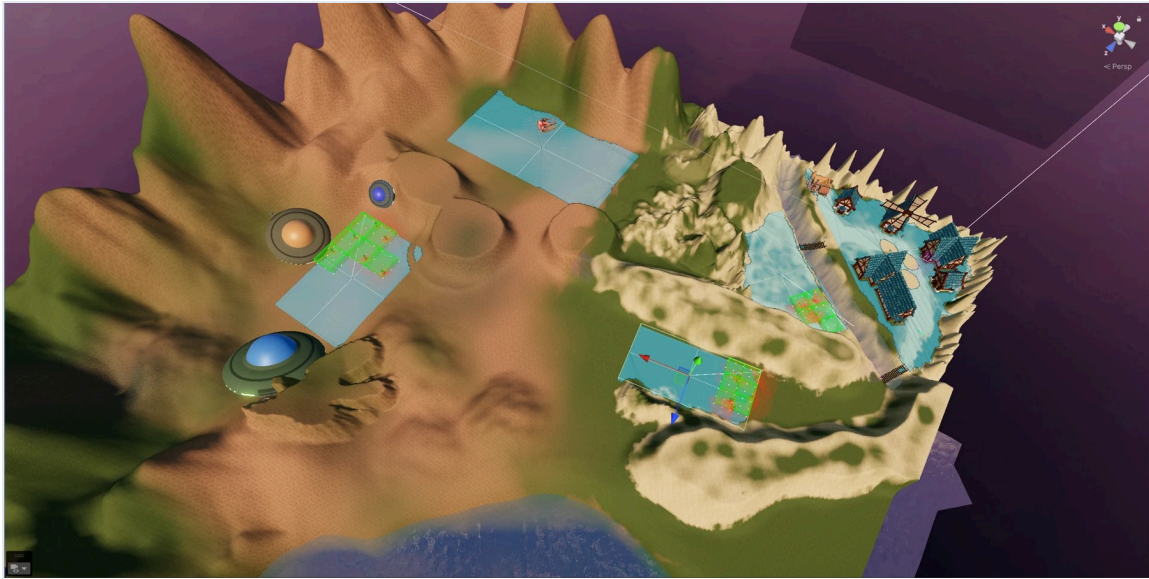
Assets/04.Scripts/Monsters/Enemies/SpawnTile.cs 발췌

CSHARP

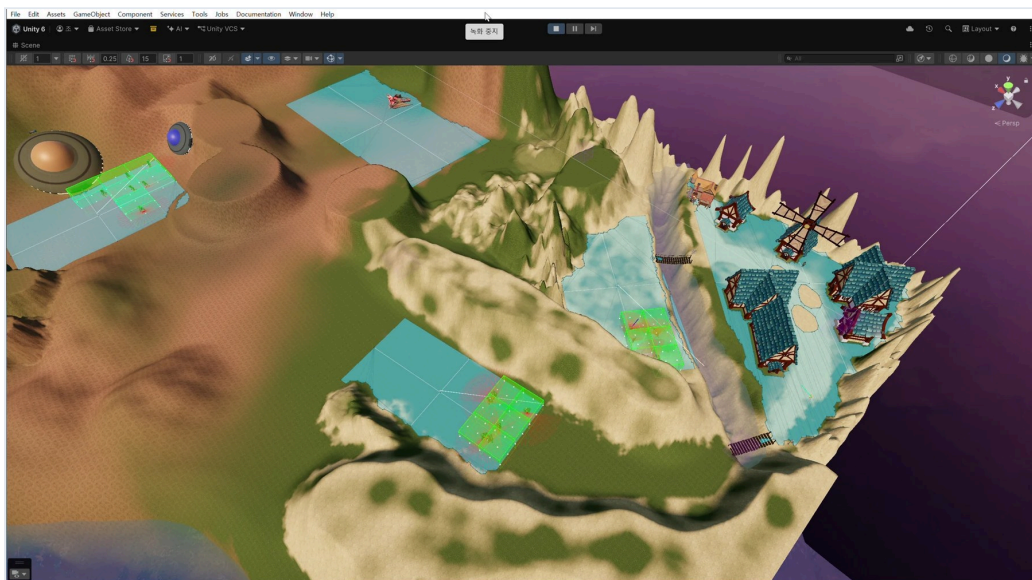
```
Gizmos.color = new Color(1f, 0f, 1f, 0.5f);
if (!_isCellFixed)
    Gizmos.color = new Color(0f, 1f, 0f, 0.5f);
var prev = Gizmos.matrix;
Gizmos.matrix = transform.localToWorldMatrix;
Gizmos.DrawCube(Vector3.zero, new Vector3(_cellWidth, _cellHeight, _cellDepth) );
if (_isCellFixed)
    Gizmos.color = Color.green;
else
    Gizmos.color = Color.purple;
Gizmos.DrawWireCube(Vector3.zero, new Vector3(_cellWidth, _cellHeight, _cellDepth));
Gizmos.matrix = prev;
```

셀 인덱스는 TextMeshPro로 표시하고 카메라를 향하게 회전한다. 확정된 셀의 순찰 지점에는 WireSphere를 그린다. 가까운 화면에서는 셀과 후보점을, 원거리에서는 영역별 배치 범위를 볼 수 있다.

스폰너 셀 배치와 박스 Gizmo · 계속



전체 맵에서 구분한 NavMesh 영역과 스폰 셀



Debug Spawner 영상에서 확인할 수 있는 영역별 박스 배치

12 / 데이터와 표현

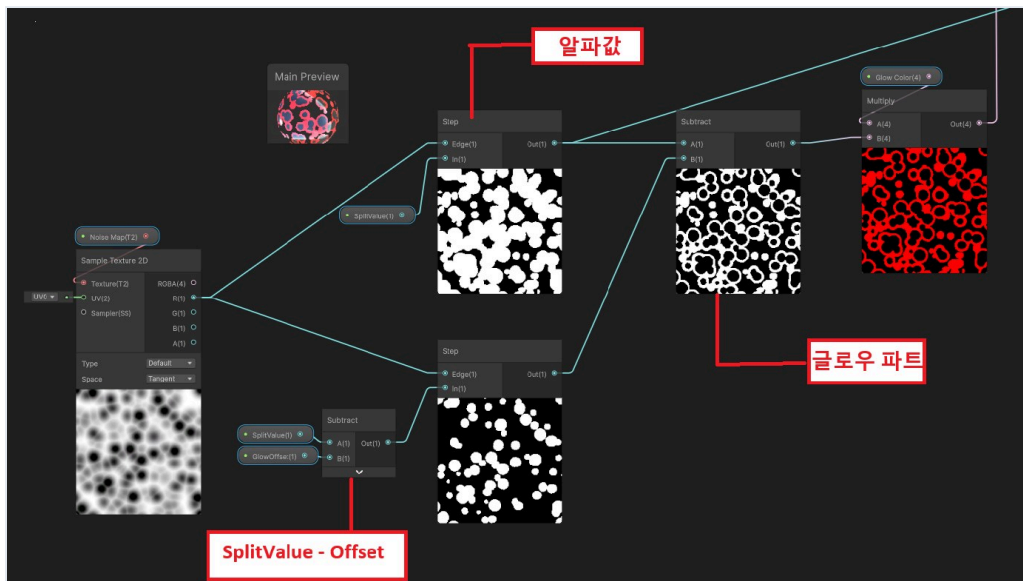
Shader Graph 디졸브와 C# 제어

노이즈 임계값과 발광 경계를 구성하고 C#에서 SplitValue를 보관해 등장과 퇴장을 표현했다.

[블로그 원문 ↗](#)

노이즈로 표시 영역 구성

Shader Graph에서 노이즈와 SplitValue를 비교해 모델의 표시 영역을 만든다. SplitValue - GlowOffset을 두 번째 기준으로 사용하고 두 임계값 사이의 영역에 발광 색을 적용한다. SplitValue는 드러나는 범위, GlowOffset은 발광 경계의 폭을 조절한다.



표시 영역과 발광 경계를 구성한 Shader Graph

디렉터리와 실행 흐름

TEXT

```
Assets/04.Scripts/
└─ Shaders/ShaderEffects.cs
```

TEXT

```
노이즈.SplitValue → 표시 영역
노이즈.SplitValue.GlowOffset → 발광 경계
DoFade → iTween → TweenOnUpdate → 머티리얼의 SplitValue
```

렌더러별 머티리얼 전환

ShaderEffects는 자식 Renderer 목록과 원본-페이지-디졸브 Material 배열을 가진다. SetNowMaterial은 eShaderEffect에 따라 각 렌더러에 사용할 머티리얼을 지정한다. 파티클처럼 Poolable에 속한 렌더러는 해당 전환에서 건너뛴다.

Shader Graph 디졸브와 C# 제어 · 계속

C#에서 임계값 보간

`DoFade`는 시작값, 종료값, 시간과 페이드 모드를 받는다. iTween의 `ValueTo`에 이 값을 넘기고 `easeInCubic` 보간을 사용한다. 갱신 콜백은 `TweenOnUpdate`, 완료 콜백은 `TweenOnComplete`로 연결한다.

[Assets/04.Scripts/Shaders/ShaderEffects.cs](#) 발췌

CSHARP

```
public void DoFade(float from, float to , float time, eFadeMode fadeMode=eFadeMode.FADE_OUT)
{
    foreach (var renderer in _renderers)
    {
        string name = renderer.material.shader.name;
        bool bo = renderer.material.HasProperty("_splitValueHash");
        _fadeMode = fadeMode;
        _isFadeEffectDone = false;
        iTween.ValueTo(gameObject, iTween.Hash(
            "from", from, "to", to, "time", time, "onupdatetarget", gameObject,
            "onupdate", "TweenOnUpdate", "oncomplete", "TweenOnComplete",
            "easetype", iTween.EaseType.easeInCubic
        ));
    }
}
```

매 갱신에서 모든 대상 렌더러의 `_SplitValue`를 같은 값으로 설정한다. 이 값이 Shader Graph의 임계값에 전달되면서 표시 영역이 시간에 따라 변한다.

[Assets/04.Scripts/Shaders/ShaderEffects.cs](#) 발췌

CSHARP

```
public void TweenOnUpdate(float value)
{
    foreach (var renderer in _renderers)
    {
        renderer.material.SetFloat("_SplitValue", value);
    }
}
```

연출 완료 처리

`TweenOnComplete`는 `_isFadeEffectDone`을 킨다. 페이즈 효과의 완료 값은 -0.5로 설정하고 `FADE_IN`이면 원본 머티리얼 배열을 다시 적용한다. 시작-종료 값과 모드에 따라 같은 컴포넌트로 등장과 퇴장 연출을 실행한다.